Scheduled Partial-Credit RL for Reliable Code Generation with Small Language Models (WIP)

Suryansh Singh Sijwali
Pennsylvania State University
University Park, USA
sss6371@psu.edu

Suman Saha
Pennsylvania State University
University Park, USA
szs339@psu.edu

Abstract

Small language models (SLMs, ≤1.5B parameters) are attractive for embedded and resource-limited development workflows because they can run under single-GPU or CPU budgets and be adapted without distributed training. SLM-based code generation is brittle under strict sandboxed evaluation, and reinforcement learning (RL) with binary test rewards is often too sparse to train SLMs reliably. This WIP paper presents a reliability-first RL framework for SLM code generation built around a joint reward. The functional term assigns intermediate credit to near-miss outcomes for syntax validity, crash-free execution, and output production, while a static-analysis term discourages unsafe shortcuts during training. On DeepSeek-Coder-1.3B evaluated on 100 stdin-style APPS+ prompts, a binary-to-partial-credit curriculum improves syntax validity to 63% and produces solutions that pass at least one test in 9% of prompts in a single generated attempt. In contrast, binary-reward PPO regresses below a supervised fine-tuning baseline and partial-credit training from scratch reaches only 27% syntax validity.

CCS Concepts: • **Computing methodologies** → **Sequential decision making**.

Keywords: Partial-Credit Reward Shaping, Reward Scheduling, Curriculum Learning, Reinforcement Learning (PPO), Small Language Models, Code Generation

ACM Reference Format:
Suryansh Singh Sijwali and Suman Saha. 2026. Scheduled Partial-Credit RL for Reliable Code Generation with Small Language Models (WIP). In *Proceedings of the 27th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '26), June 15–16, 2026, Boulder, CO, USA.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3814943.3816167

1 Introduction

Code-capable large language models (LLMs) are widely used as programming assistants. Yet producing code that behaves reliably under strict execution environments remains difficult, especially when solutions must run in sandboxes with limited time, memory, and dependency access. Empirical studies report that LLM-generated code can include security-sensitive patterns [13]. A controlled user study further found that developers using AI assistants wrote more vulnerable code while reporting higher confidence in its security [14]. These findings motivate training and evaluation pipelines that emphasize reliable execution and discourage insecure shortcuts, rather than optimizing only for fluent code.

A key practical constraint in embedded and resource-limited development is deployment under tight compute and memory budgets. Concrete deployment contexts include offline pre-commit hooks in regulated build pipelines where source cannot leave the network boundary, air-gapped firmware-development workflows that prohibit cloud-API calls, and local CI assistants on developer laptops in bandwidth-limited or intermittent-connectivity environments. These contexts require models that run within single-GPU or CPU inference limits and can be adapted without distributed training infrastructure. This motivates focusing on *small language models* (SLMs, ≤1.5B parameters). SLMs are practical to fine-tune and deploy in these environments, but they are also more brittle: they produce syntax errors, runtime failures, and incomplete outputs more often than larger models.

Most existing approaches that aim to improve code generation quality either intervene at inference time (for example, prompting or constrained decoding [5]) without changing the underlying policy, or rely on training-time methods such as learned steering/prefix tuning (e.g., SVEN [6]) or instruction tuning on security datasets (e.g., SafeCoder [7]). Reinforcement learning provides a complementary path by using execution feedback as a training signal. Prior work such as CodeRL [10], PPOCoder [16], StepCoder [2], and RLTF [11] shows that RL with execution-based feedback can improve test performance. However, existing pipelines are typically evaluated with binary rewards tied to full test success. For SLMs, that signal is often too sparse to support stable improvement. In addition, when training optimizes only for passing tests, models may learn brittle shortcuts that satisfy shallow test suites but are undesirable in practice, including patterns that raise security concerns.

This paper presents a WIP reinforcement learning framework that targets *reliability-first* training of small language

LCTES '26, Boulder, CO, USA

ACM ISBN 979-8-4007-2721-4/2026/06
https://doi.org/10.1145/3814943.3816167

models (SLMs) for code generation. We introduce a *partial-credit* functional reward that distinguishes near-miss outcomes and assigns intermediate credit for milestones such as producing syntactically valid code, executing without crashing, and producing output. We also include a static-analysis based security term in the overall objective to discourage unsafe shortcuts during training. In this WIP, we focus on reliability metrics because stdin-style APPS+ prompts in our Python evaluation rarely produce differentiating security findings. We demonstrate the approach on DeepSeek-Coder-1.3B using Python code generation on APPS+.

This WIP paper contributes: (1) A resource-aware RL framework for SLM code generation (≤1.5B) designed for single-GPU or CPU deployment constraints. (2) A joint training objective $R = \alpha R_{\text{func}} + \beta R_{\text{sec}}$ that incorporates static-analysis feedback as a safety guardrail. (3) A partial-credit functional reward that assigns staged credit for syntax validity, execution without crash, and output production to mitigate reward sparsity in SLM training. (4) Preliminary results on 100 APPS+ prompts that identify reward scheduling as the operative factor. A binary-to-partial-credit curriculum produces stable training gains, while binary-reward PPO regresses below supervised fine-tuning (SFT) and partial-credit training from scratch underperforms the curriculum.

2 Related Work

This section situates our WIP contribution on scheduled partial-credit reinforcement learning for reliable code generation in small language models within three threads. We focus on work that uses execution feedback for training, and on techniques that constrain insecure behavior during generation. We keep the discussion brief and selective, since this paper's evaluation targets reliability in a strict sandboxed setting.

2.1 RL for Code Generation

CodeRL [10] introduced execution-based rewards for code generation using unit-test outcomes, showing that reinforcement learning can improve behavior beyond supervised fine-tuning. PPOCoder [16] applies PPO directly to code synthesis with execution outcomes as reward and reports gains on benchmarks such as HumanEval and APPS. StepCoder [2] addresses sparse feedback by providing intermediate signals via subtask decomposition, which aligns with our partial-credit reward design. RLTF [11] supports fine-grained unit-test feedback over a single pass or fail signal. In contrast to prior RL-for-code work that targets larger backbones, our WIP studies the small-model regime and finds that binary execution rewards regress below the SFT baseline, which motivates scheduled partial-credit shaping as the load-bearing design choice for stable training.

2.2 Security-Constrained Code Generation

Several approaches encourage secure code generation without relying on execution-based RL. SVEN [6] steers generation using learned continuous prompt vectors, enabling control over security-related tendencies without updating the base model weights. SafeCoder [7] uses instruction tuning with security-focused data to reduce vulnerable patterns while maintaining utility on general tasks. CodeGuard+ [5] uses constrained decoding guided by static analysis at inference time, which can block insecure generations without changing training. These methods motivate our use of static analysis as a constraint term in the training objective. Our main novelty in this WIP is the scheduled partial-credit shaping that targets reliability under strict sandboxed evaluation.

2.3 Embedded Systems Code Generation

For embedded and resource-constrained workflows, the practical challenge is not only correctness but also reliability under limited compute, memory, and restricted execution environments. Englhardt et al. [4] report that models often violate hardware- or environment-specific constraints even when output is syntactically valid, highlighting gaps that matter in device-facing toolchains. Dunne et al. [3] analyze weakness patterns in LLM-generated C code for embedded networking contexts, reinforcing the need for security-aware constraints in future extensions. Our current WIP evaluation uses Python problems to isolate reliability and reward sparsity issues at small scale, and we position security-focused and embedded-focused evaluation as the natural next step.

3 Method Overview

This section presents the reinforcement learning objective and reward design used in our WIP study. We focus on reliability under strict sandboxed evaluation for small language models, where generations often fail due to syntax errors, crashes, or missing output. Figure 1 summarizes the workflow. We then define the combined reward objective and partial-credit functional reward used in PPO, and clarify how the dataset is used for training and evaluation.

Dataset and usage: We use the stdin-style APPS+ benchmark [8], which contains 7,408 programming problems. Each prompt specifies a programming task with a fixed input / output interface, and evaluation runs the generated Python program against hidden tests in a sandbox. In our setup, APPS+ is used for supervised fine-tuning and PPO training, and we also hold out 100 prompts that are used only for evaluation.

3.1 Combined Reward Objective

Given a prompt x, the policy π_θ generates a program $y \sim \pi_\theta(\cdot \mid x)$. Training optimizes a scalar reward

$$R(y) = \alpha \cdot R_{\text{func}}(y) + \beta \cdot R_{\text{sec}}(y), \quad \alpha = 0.6,\ \beta = 0.4. \quad (1)$$

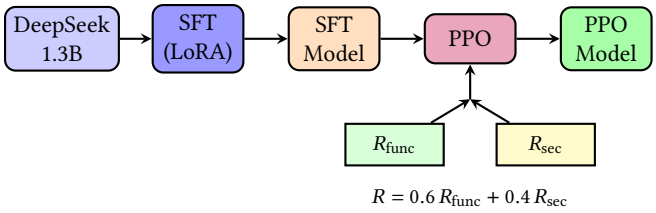

Figure 1. Overview of the training workflow. Supervised fine-tuning (SFT) with LoRA initializes the policy, and PPO optimizes the joint reward. R_{func} is the staged partial-credit functional reward in Table 1. $R_{\text{sec}} = 1 - V$ penalizes severity-graded static-analysis findings.

The weights are chosen so that a HIGH-severity security finding reduces R by $\beta \cdot V_{\text{HIGH}} = 0.4$, which exceeds the largest gain R_{func} can contribute to R in a single step ($\alpha \cdot 0.4 = 0.24$, from producing output to passing every test in Table 1). Unsafe shortcuts therefore remain a net-negative gradient even at the largest possible functional reward step. A sensitivity sweep over (α, β) is deferred to the extended study because the current APPS+ evaluation yields $R_{\text{sec}} = 1.0$ across variants (Section 4). The functional term R_{func} measures progress toward a correct, runnable solution under the sandboxed judge. The security term R_{sec} discourages unsafe shortcuts during learning by penalizing static-analysis findings. In the current Python evaluation we use Bandit[1] restricted to MEDIUM and HIGH severity, and we exclude LOW findings such as benign `input()` usage required by APPS+'s stdin format. We define a normalized severity score $V \in [0, 1]$ with HIGH= 1.0 and MEDIUM= 0.5, and set

$$R_{\text{sec}} = 1 - V, \tag{2}$$

so that clean code yields $R_{\text{sec}} = 1.0$ and higher-severity findings reduce the reward linearly.

3.2 Partial-Credit Functional Reward

Binary functional rewards assign $R_{\text{func}} = 1$ only when all tests pass and 0 otherwise, which is highly sparse for SLMs on strict judge-style tasks. Many generations are near misses that fail for predictable reasons, including syntax errors, runtime crashes, or producing no output. Our partial-credit functional reward makes these failure modes distinguishable by assigning intermediate credit for milestones needed for reliable execution. This staged design follows potential-based reward shaping [12], which provides intermediate milestones while preserving the optimal policy under the final task objective. Table 1 defines the scoring with staged credit for parsing, crash-free execution, and output production. The final tier allocates the remaining credit proportional to the fraction of tests passed.

[1] https://bandit.readthedocs.io/

Table 1. Partial-credit R_{func}: staged scoring. T = total tests, k = tests passed.

Stage	Condition	R_{func}
0	Syntax error / not parseable	0.0
1	Valid syntax	0.2
2	Executes without crash	0.4
3	Produces any stdout	0.6
4	Passes k of T tests	$0.6 + 0.4 \cdot k/T$

Table 2. Baseline benchmark on stdin-style APPS+ (zero-shot, our runs). *Test Pass* = all tests pass; *Sec. Clean* = no Bandit MEDIUM/HIGH findings.

Model	Syntax Valid	Test Pass	Sec. Clean
DeepSeek-6.7B[2]	83.4%	14.3%	96.3%
CodeLlama-7B[3]	48.9%	7.6%	99.5%
StarCoder2-7B[4]	20.2%	1.3%	99.7%

4 Experiments and Preliminary Results

This section summarizes our experimental setup and preliminary results on stdin-style APPS+ in a strict sandboxed evaluation setting. We first report a small zero-shot baseline comparison, where zero-shot means a single generated sample with no in-context examples, to contextualize the capability gap between larger code models and the 1.3B target. We then describe the training setup for the 1.3B model and evaluate three PPO variants to isolate the effect of partial credit and its scheduling.

4.1 Setup

We initialize from DeepSeek-Coder-1.3B-Instruct[2] and adapt it with LoRA [9] (rank r=16, α=32; 6.3M trainable parameters, 0.47% of total). We use the stdin-style APPS+ corpus [8] (7,408 problems; each capped at 5 test cases). We run supervised fine-tuning (SFT) for 3 epochs with cross-entropy loss to initialize PPO [15]. PPO uses a KL penalty (λ=0.1) against the SFT reference to discourage reward hacking.

4.2 Baselines and PPO Variants

To contextualize the reliability gap across scales, Table 2 reports a zero-shot baseline on APPS+ for three 7B-class reference models. We treat these as upper-bound references rather than deployment targets. For our main study, we compare the 1.3B SFT baseline against three PPO variants. *PPO-simple* uses a binary functional reward and initializes from SFT. *PPO-fresh* uses partial credit and initializes from SFT.

[2] https://arxiv.org/abs/2401.14196
[3] https://arxiv.org/abs/2308.12950
[4] https://arxiv.org/abs/2402.19173

Table 3. Evaluation on 100 APPS+ prompts (~85 held-out). ≥1-P = at least one test passed; All-P = all tests pass; $\overline{R}$ = mean joint reward. Bootstrap CIs: ±10% (Syn.), ±6% (≥1-P). PPO-simple and PPO-fresh are statistically indistinguishable on syntax validity (±10% overlap); the near-uniform $\overline{R}$ reflects R_{sec}=1.0 for all variants (see text).

Model	Syn.%	≥1-P%	All-P%	$\overline{R}$
SFT Baseline	44.0	3.0	1.0	0.40
PPO-simple (binary)	18.0	0.0	0.0	0.38
PPO-fresh (partial)	27.0	2.0	0.0	0.40
PPO-cont. (partial)	**63.0**	**9.0**	**2.0**	**0.42**

PPO-continue uses partial credit and initializes from the PPO-simple checkpoint.

4.3 Training Protocol

Each PPO variant is trained for 500 episodes with batch size 2 (learning rate 10^{-6}, clip range ε=0.2, 4 PPO epochs per batch, temperature 0.7), covering about 13.5% of the APPS+ corpus on a single NVIDIA V100 16 GB. We evaluate on 100 randomly sampled APPS+ prompts. Training covers about 1,000 unique prompts over 500 episodes, so the expected overlap with the evaluation set is about 15 prompts, leaving roughly 85 held-out examples.

4.4 Preliminary Results

Table 3 summarizes results on the 100-prompt set. The scale gap is immediate. DeepSeek-6.7B achieves 83.4% syntax validity zero-shot (Table 2) whereas the 1.3B SFT baseline reaches 44%. Scheduling the reward from binary to partial credit improves syntax validity to 63%, which we do not observe with binary reward at this scale. PPO-simple degrades syntax validity below SFT (18% vs. 44%), consistent with sparse feedback destabilizing a 1.3B policy that cannot bridge near-miss generations to correct solutions. PPO-continue (Table 3) shows that this degradation can be recovered by inheriting PPO-simple's exploration before partial credit applies.

Security analysis. Bandit reported no findings across 400 samples (100 prompts × 4 variants). This null result likely reflects APPS+'s algorithmic focus rather than security improvement, so R_{sec}=1.0 for all variants and $\overline{R}$ provides little differentiation in Table 3. Evaluating security effects will require vulnerability-eliciting prompts, which we plan to include in an extended study.

Binary reward (PPO-simple) degrades performance relative to SFT, consistent with sparse feedback on near-miss programs. PPO-fresh does not surpass PPO-simple (27% vs. 18%, within overlapping CIs), suggesting that partial-credit reward alone cannot overcome instability without an RL warmup phase. PPO-continue performs best. The gap between PPO-fresh and PPO-continue (27% vs. 63%) isolates the warmup effect, because both variants use the same partial-credit reward but only PPO-continue inherits PPO-simple's trajectory exploration before partial credit applies. This is consistent with curriculum learning [1] and motivates reward scheduling as a key design concern for SLM training.

5 Future Work

This WIP identifies reward scheduling as a central design choice for stabilizing RL training of small code models under strict sandboxed evaluation. A binary-to-partial-credit curriculum is the most reliable configuration we tested. Next, we will scale to additional model families and a larger held-out APPS+ evaluation, and extend beyond algorithmic prompts to security-focused benchmarks and out-of-distribution prompts that produce meaningful vulnerability signals and test generalization. We note that the held-out evaluation reported here stays within the APPS+ distribution, so broader generalization claims will require cross-benchmark evaluation such as MBPP or HumanEval alongside the security-focused corpora above. We will also design fine-grained security partial credit by mapping static-analysis findings to a vulnerability taxonomy (e.g., CWE) and rewarding staged improvements from removing high-severity issues to later refinements. Finally, we will extend the framework to C code generation for embedded toolchains by integrating C-focused analyzers and evaluating under resource-constrained compilation and execution settings.

6 Conclusion

We presented a WIP reinforcement learning framework for reliability-first code generation with small language models. The key idea is a partial-credit functional reward that distinguishes common near-miss failures and provides intermediate milestones toward runnable, test-passing solutions. On DeepSeek-Coder-1.3B evaluated on 100 stdin-style APPS+ prompts, binary-reward PPO regresses below the SFT baseline, and only a binary-to-partial-credit curriculum produces stable gains in syntax validity and partial test success. This regression identifies reward sparsity as a primary failure mode for RL training of small code models, and reward scheduling as a central design choice. The reported pass rates are not sufficient for autonomous use but they establish a stable training regime that motivates a broader study targeting security-taxonomy-aligned rewards and embedded C workloads.

References

[1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum Learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09)*. ACM, Montreal, Quebec, Canada, 1–8. doi:10.1145/1553374.1553380

[2] Shihan Dou, Yan Liu, Haoxiang Jia, et al. 2024. *StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback*. arXiv:2402.01391 doi:10.48550/arXiv.2402.01391

[3] Murray Dunne, Kylee Schram, and Sebastian Fischmeister. 2024. Weaknesses in LLM-Generated Code for Embedded Systems Networking. In *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS '24)*. IEEE, Washington, DC, USA, 250–261.

[4] Zachary Englhardt, Richard Li, Dilini Nissanka, et al. 2024. Exploring and Characterizing Large Language Models for Embedded System Development and Debugging. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*. ACM, New York, NY, USA, Article 150, 9 pages. doi:10.1145/3613905.3650764

[5] Yanjun Fu, Ethan Baker, Yu Ding, and Yizheng Chen. 2024. *Constrained Decoding for Secure Code Generation*. arXiv:2405.00218 doi:10.48550/arXiv.2405.00218

[6] Jingxuan He and Martin Vechev. 2023. Large Language Models for Code: Security Hardening and Adversarial Testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. ACM, New York, NY, USA, 1865–1879.

[7] Jingxuan He, Mark Vero, Gabriela Krasnopolska, and Martin Vechev. 2024. Instruction Tuning for Secure Code Generation. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML '24)*. JMLR.org, Article 723, 20 pages.

[8] Dan Hendrycks, Steven Basart, Saurav Kadavath, et al. 2021. Measuring Coding Challenge Competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS D&B '21, Vol. 1)*. Curran Associates, Inc.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR '22)*. OpenReview.net.

[10] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1549, 15 pages.

[11] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, et al. 2023. RLTF: Reinforcement Learning from Unit Test Feedback. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=hjYmsV6nXZ

[12] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 278–287.

[13] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *Commun. ACM* 68, 2 (2025), 96–105. doi:10.1145/3610721

[14] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. ACM, New York, NY, USA, 2785–2799.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347 doi:10.48550/arXiv.1707.06347

[16] Parshin Shojaee, Aneesh Jain, Sindhu Tipirneni, and Chandan K. Reddy. 2023. Execution-based Code Generation using Deep Reinforcement Learning. *Transactions on Machine Learning Research* (2023).